

Milestone 2 完了サマリー

このドキュメントについて

このドキュメントは、**Milestone 2(基盤構築フェーズ)の完了報告書**です。4つの基盤システムの実装状況、技術スタック、Phase 3への準備状況を簡潔にまとめています。

対象読者: クライアント、プロジェクトマネージャー、開発者、技術リード

難易度: 初級～中級(技術的な詳細は各詳細ドキュメントを参照)

推定読了時間: 5-10分

実装状況

Milestone 2の4つの基盤機能をすべて完全実装しました。

✓ 基盤1: アカウント管理データベース - 200アカウント管理 - パスワード暗号化(Fernet/AES-256-GCM) - 自動エラー処理(5回で無効化) - Supabase (PostgreSQL Online) 統合	
✓ 基盤2: ログ基盤 - 全操作の自動記録 - コード位置自動検出 - 4つの復旧戦略 - リアルタイム配信(WebSocket)	
✓ 基盤3: ブラウザ自動化基盤(Playwright) - 1アカウント = 1コンテキスト(完全分離) - 自動ログイン・URL自動判別 - 手動操作検知・クラッシュ復旧 - 並列実行(最大10ワーカー)	
✓ 基盤4: 競争対策モジュール - ランダム遅延・分散スケジューリング - リソース管理・優先度制御 - レート制限・競合検知 - バッチ実行	
総合進捗: 100% (4/4基盤が実装完了)	

主要実装内容の詳細

基盤1: アカウント管理データベース

- 200アカウント: UUID、ユーザー名、店舗名、ステータス管理
- 暗号化: Fernet (AES-256-GCM) による強固なパスワード暗号化
- 自動無効化: エラー5回連続で自動的にアカウントを無効化
- リレーション: ログ、統計、セッション履歴と1:N関係
- インデックス: 高速検索のための最適化済みインデックス

基盤2: ログ基盤

- 自動記録: すべての操作を自動的にログ記録
- コード位置検出: ファイル名、関数名、行番号を自動記録
- 復旧システム: ログから過去の成功状態に復旧可能
- リアルタイム: WebSocketでフロントエンドにリアルタイム配信
- トレーシング: リクエストIDによる操作の追跡

基盤3: ブラウザ自動化基盤

- コンテキスト分離: 各アカウント専用のブラウザコンテキスト

- **自動ログイン**: パスワード復号化と自動フォーム入力
- **URL自動判別**: 複数URLから自動選択とキャッシュ
- **手動操作検知**: トークン監視とDOM変化検知
- **クラッシュ復旧**: アプリクラッシュの自動検知と再起動
- **並列実行**: 安全な並列実行(最大10ワーカー)

基盤4: 競争対策モジュール

- **タイミング制御**: 0-30秒のランダム遅延
- **分散実行**: 200アカウントを3時間に分散
- **リソース管理**: CPU/メモリ監視と動的制御
- **優先度管理**: エラー回数・最終成功時刻による自動優先度
- **レート制限**: API呼び出しを10回/分に制限
- **バッチ実行**: ランダム順序でバッチ実行

📦 技術スタック

現在のシステム構成:

レイヤー	技術
フロントエンド	TypeScript + React
バックエンド	Python + Flask
データベース	Supabase (PostgreSQL Online)
自動化	Playwright

```
TypeScript (フロントエンド)
  ↳ HTTP/WebSocket
Python (バックエンド)
  ↳ SQL
Supabase (PostgreSQL Online)
```

✅ Phase 3 実行準備完了

Phase 3(業務ロジック実装)を今すぐ開始できる状態です。

準備完了項目

- ✅ **データベース**: Supabaseで200アカウント管理可能
- ✅ **API基盤**: Python/Flaskで完全実装済み
- ✅ **UI**: TypeScript/Reactで実装済み
- ✅ **自動化エンジン**: Playwright完全統合済み
- ✅ **ログ・監視**: 全操作記録・復旧機能実装済み
- ✅ **競争対策**: 検知回避機能実装済み

Phase 3で実装する内容

Phase 3では、構築済みの基盤の上に**業務ロジックのみ**を実装します:

1. **スケジュール更新処理**
 - 営業日の自動設定
 - 翌日の出勤設定
 - スケジュール保存
2. **待機接客配信制御**
 - 営業アカウント数の確認
 - 退勤時刻の監視
 - 待機設定の自動適用

予定期間: 3-4週間

📊 開発実績

実装したコンポーネント数

カテゴリ	コンポーネント数	主要ファイル数
データベース	8テーブル + 3ビュー	15+ ファイル
バックエンドAPI	20+ エンドポイント	25+ ファイル
自動化モジュール	7モジュール	10+ ファイル
フロントエンド	15+ コンポーネント	20+ ファイル
合計	50+ コンポーネント	70+ ファイル

コード統計

- バックエンド(Python): 約5,000行
- フロントエンド(TypeScript): 約3,000行
- データベーススキーマ(SQL): 約500行
- 設定ファイル(YAML/JSON): 約300行
- 合計: 約8,800行

実装した主要機能

- ☒ データベース: 8テーブル、3ビュー、複数インデックス
- ☒ 暗号化: Fernet暗号化・復号化システム
- ☒ ログ記録: 自動コード位置検出システム
- ☒ 復旧機能: 4つの復旧戦略
- ☒ ブラウザ自動化: Playwright完全統合
- ☒ 並列実行: ThreadPoolExecutorによる並列処理
- ☒ 競争対策: 6つの対策機能
- ☒ リアルタイム通信: WebSocket統合
- ☒ API: RESTful API 20+ エンドポイント
- ☒ UI: React + TypeScript 15+ コンポーネント

🔗 結論

Milestone 2達成事項

- ☒ 4つの基盤すべてを完全実装
- ☒ TypeScript + Python + Supabaseの技術スタック構築
- ☒ 200アカウント対応の堅牢な基盤完成
- ☒ Phase 3実行準備100%完了

次のステップ

Phase 3を即座に開始できます。

すべての技術的基盤が整っており、業務ロジックの実装のみに集中できる状態です。

📖 詳細ドキュメント

各基盤の技術的な詳細については、以下のドキュメントをご参照ください:

基盤1: アカウント管理データベース

 [account_database_detailed_jp.md \(/account_database_detailed_jp.md\)](#)

- データベース設計の詳細
- 暗号化システムの実装
- フロントエンド統合
- パフォーマンス最適化

基盤2: ログ基盤

 [logging_infrastructure_detailed_jp.md \(/logging_infrastructure_detailed_jp.md\)](#)

- 自動コード位置検出の仕組み
- コンテキストマネージャーパターン
- 4つの復旧戦略
- リアルタイムログ配信

基盤3: ブラウザ自動化基盤(Playwright)

 [browser_automation_detailed_jp.md \(./browser_automation_detailed_jp.md\)](#)

- 1アカウント = 1コンテキストの原則
- 自動ログインの実装
- 手動操作検知・クラッシュ復旧
- 並列実行エンジン

基盤4: 競争対策モジュール

 [competitive_countermeasure_detailed_jp.md \(./competitive_countermeasure_detailed_jp.md\)](#)

- 実行タイミング制御
- リソース管理・優先度管理
- レート制限・競合検知
- パッチ実行システム

報告日: 2025年12月26日

ステータス: Milestone 2完了、Phase 3実行準備完了

ドキュメントバージョン: 2.0